

Praca Dyplomowa Inżynierska

Dominik Kubicki 210882
Jakub Kindlik 210871

System automatycznego sterowania rakieta oparty na symulacjach komputerowych i algorytmach sztucznej inteligencji

Praca dyplomowa na kierunku
Informatyka

Praca wykonana pod kierunkiem
dra inż. Pawła Hosera
Instytut Informatyki Technicznej
Katedra Sztucznej Inteligencji

Warszawa, rok 2025



SZKOŁA GŁÓWNA
GOSPODARSTWA
WIEJSKIEGO

Wydział Zastosowań
Informatyki
i Matematyki

Oświadczenie Promotora pracy

Oświadczam, że niniejsza praca została przygotowana pod moim kierunkiem i stwierdzam, że spełnia ona warunki do przedstawienia tej pracy w postępowaniu o nadanie tytułu zawodowego.

Data

Podpis promotora

Oświadczenie autora pracy

Świadom/a odpowiedzialności prawnej, w tym odpowiedzialności karnej za złożenie fałszywego oświadczenia, oświadczam, że niniejsza praca dyplomowa została napisana przeze mnie samodzielnie i nie zawiera treści uzyskanych w sposób niezgodny z obowiązującymi przepisami prawa, w szczególności z ustawą z dnia 4 lutego 1994 r. o prawie autorskim i prawach pokrewnych (Dz. U. 2019 poz. 1231 z późn. zm.)

Oświadczam, że przedstawiona praca nie była wcześniej podstawą żadnej procedury związanej z nadaniem dyplomu lub uzyskaniem tytułu zawodowego.

Oświadczam, że niniejsza wersja pracy jest identyczna z załączoną wersją elektroniczną. Przyjmuję do wiadomości, że praca dyplomowa poddana zostanie procedurze antyplagiatowej.

Data

Podpis autora pracy

Streszczenie

Temat

Streszczenie

Słowa kluczowe – Słowa kluczowe

Summary

Temat ang

Streszczenie ang

Keywords – Słowa kluczowe ang

Spis treści

1	Wstęp	9
1.1	Cel pracy	9
1.2	Struktura pracy	9
1.3	Użyte technologie	10
1.4	Przegląd dostępnych rozwiązań.	11
1.4.1	Silniki symulacji lotów raketowych.	11
1.4.2	Algorytmy uczenia maszynowego oparte o RL.	14
1.4.3	Podsumowanie.	16
2	Moduł I: Silnik fizyczny symulacji lotów raketowych.	18
2.1	Założenia projektowe.	18
2.2	Teoretyczne podstawy fizyczne.	19
2.2.1	Opis sił działających na rakietę	20
2.2.2	Równania ruchu postępowego	21
2.2.3	Ruch obrotowy i momenty sił	22
2.2.4	Modelowanie środowiska atmosferycznego	23
2.2.5	Kąt natarcia rakiety	23
2.2.6	Geometria rakiety	24
2.2.7	Obroty układów współrzędnych	26
2.2.8	Podsumowanie	27
2.3	Użyte technologie.	27
2.4	Projekt silnika fizycznego.	29
2.4.1	Architektura systemu symulacji	29
2.4.2	Interfejsy	30
2.4.3	Klasy	36
2.4.4	Proces symulacji.	36
2.4.5	Konfiguracja symulacji (Setup)	36
2.4.6	Symulacja	37

2.4.7	Uzasadnienie przyjętego rozwiązania	39
2.5	Model sił działających na raketę	40
2.5.1	Funkcje pomocnicze	42
2.5.2	Siły działające na raketę	43
2.5.3	Obliczanie zmian w ruchu rakiety	47
2.5.4	Podsumowanie	49
2.6	Moduł wizualizacyjny symulacji.	50
2.7	Dokładność symulacji.	50
2.8	Ewolucja projektu i wnioski.	50
3	Moduł II: Algorytm sterujący raketą.	51
3.1	Teoretyczne podstawy.	51
3.2	Użyte technologie.	51
3.3	Wybór odpowiedniego algorytmu uczenia maszynowego.	51
3.4	Zbieranie danych treningowych.	51
3.5	Proces uczenia i optymalizacja.	51
3.6	Ewolucja projektu i wnioski.	51
4	Integracja komponentów: Silnik fizyczny i sztuczna inteligencja	52
4.1	Interakcja między silnikiem fizycznym a modelem AI	52
4.2	Synchronizacja działania obu modułów	52
4.3	Rozwój interfejsu użytkownika	52
5	Wyniki działania i wnioski	53
5.1	Ewaluacja działania silnika fizycznego	53
5.2	Ocena skuteczności modelu sztucznej inteligencji	53
5.3	Analiza osiągniętych rezultatów i ich implikacje	53
6	Podsumowanie	54
6.1	Podsumowanie osiągnięć	54
6.2	Wnioski końcowe	54
6.3	Perspektywy rozwoju i dalsze badania	54
	Bibliografia	55

1 Wstęp

1.1 Cel pracy

Celem niniejszej pracy inżynierskiej jest stworzenie systemu sterującego, opartego na algorytmach samouczących, zdolnego do przeprowadzenia stabilnego, pionowego lotu rakiety o naturalnie niestabilnej charakterystyce. W celu spełnienia tych założeń konieczne było stworzenie środowiska, pozwalającego na wielokrotne prowadzenie symulacji lotu rakiety i trenowanie algorytmu sterującego. Stworzenie silnika fizycznego spełniającego te wymagania jest pośrednim celem niniejszej pracy.

1.2 Struktura pracy

Niniejsza praca inżynierska powstała w wyniku kolaboracji dwóch współtwórców, dlatego też podzielona jest ona na dwa główne moduły.

- Moduł I: Silnik fizyczny symulacji lotów raketowych. Autor: Jakub Kindlik;
- Moduł II: Algorytm sterujący rakieta. Autor: Dominik Kubicki;

Główne moduły pracy są ze sobą tak ściśle związane, że aby zachować spójność, prace inżynierskie dwóch autorów powstały jako jeden dokument. Dzięki temu możliwe jest utrzymanie ciągu logicznego, prowadzącego od powstania silnika fizycznego przez trenowanie na nim modelu sterującego aż do integracji obu systemów i wniosków końcowych.

Każdy z modułów jest pełną pracą inżynierską samą w sobie, zawierając podstawy teoretyczne, przedstawienie wykorzystanych technologii, opis i dekonstrukcje stworzonego rozwiązania, wnioski, wyniki i opis procesu powstawania.

Łącznie oba projekty-moduły tworzą kompletny system sterujący rakieta, wytrenowany na silniku fizycznym, co spełnia założony cel pracy.

Pozostałe rozdziały pracy poświęcone są opisowi kwestii dotyczących obu modułów pracy inżynierskiej, takich jak między innymi wstęp, integracja obu części projektu, końcowe wnioski wyniki oraz podsumowanie całości rozwiązania pracy.

1.3 Użyte technologie

Poszczególne biblioteki i narzędzia wykorzystane podczas pracy nad każdym z modułów zostały opisane dedykowanych tym modułom rozdziałach pracy. Jednakże niektóre technologie oraz narzędzia zostały wykorzystane podczas pracy nad obydwooma modułami projektu, tworzonymi w ramach niniejszej pracy inżynierskiej. Są to:

- **Git oraz Github** - narzędzia wykorzystane do kontroli wersji projektu. Umożliwiły one wspólną pracę nad kodem projektu oraz dostęp do archiwalnych wersji projektu. Git oraz Github zostały wybrane ze względu na ich popularność, oraz wsparcie dla wielu języków, a także na znajomość tych narzędzi przez autorów pracy.
- **Python 3** - język programowania, w którym napisane zostały obydwa moduły projektu. Python został wybrany jako język programowania w projekcie ze względu na jego prostotę oraz znajomość tego języka przez autorów pracy. Ważnym czynnikiem wpływającym na wybór tego języka jest również popularność Pythona w dziedzinie uczenia maszynowego oraz sztucznej inteligencji. Dzięki wykorzystaniu tego języka do obu modułów projektu ich integracja była znacznie prostsza.
- **PyCharm Professional** - środowisko programistyczne wykorzystane do tworzenia kodu projektu. PyCharm został wybrany ze względu na dedykowane wsparcie dla języka Python, w którym napisany został projekt, oraz na znajomość tego narzędzia przez autorów pracy.

Ponadto, do tworzenia pracy inżynierskiej wykorzystane zostały następujące technologie umożliwiające stworzenie i edycję niniejszego dokumentu:

- **LaTeX** - system składu tekstu wykorzystany do napisania niniejszego dokumentu. LaTeX został wybrany ze względu na jego popularność wśród naukowców oraz inżynierów, prostotę tworzenia pracy inżynierskiej w tym narzędziu oraz znajomość tego narzędzia przez autorów pracy.
- **SGGW-thesis** - biblioteka do tworzenia pracy inżynierskiej w zgodzie z wymaganiami formatowania pracy inżynierskiej na SGGW. Biblioteka ta została wybrana ze względu na jej prostotę użycia oraz zgodność z wymaganiami formatowania pracy inżynierskiej na SGGW.

Zastosowanie tych narzędzi pozwoliło na efektywną pracę nad projektem oraz

na stworzenie tej pracy inżynierskiej. Narzędzia te są szeroko dostępne i większości darmowe, co pozwoliło na ich wykorzystanie w ramach pracy inżynierskiej.

1.4 Przegląd dostępnych rozwiązań.

Dziedzina hobbystycznej symulacji lotów raketowych, choć niszowa w kontekście informatyki, inżynierii i fizyki, posiada grono entuzjastów oraz specjalistów, którzy przez lata rozwijali narzędzia i algorytmy umożliwiające symulację takich lotów. Istnieje również duży rynek komercyjnych narzędzi stosowanych przez przedsiębiorstwa oraz wojsko do symulacji lotów raketowych. Systemy te charakteryzują się wysoką dokładnością, złożonością i są zazwyczaj niedostępne dla hobbystów oraz użytkowników prywatnych. W związku z tym, w niniejszym rozdziale opisane zostały jedynie rozwiązania ogólnodostępne. Poniżej przedstawiono narzędzia i technologie, które umożliwiają symulację lotów raketowych, a także gotowe silniki fizyczne oraz algorytmy uczenia maszynowego, mogące zostać zastosowane w tym kontekście.

Znacznie bardziej popularna dziedzina uczenia maszynowego, oparta na Reinforcement Learning (RL), oferuje liczne rozwiązania, które mogą zostać wykorzystane do trenowania modeli sztucznej inteligencji odpowiedzialnych za sterowanie lotem rakiety. W tym rozdziale przedstawione zostały najczęściej wykorzystywane z nich — zarówno biblioteki służące do trenowania modeli, jak i narzędzia umożliwiające przeprowadzanie treningów na symulacjach.

Dobra znajomość dostępnych na rynku rozwiązań pozwala na wybór najlepszego narzędzia do konkretnego projektu, a także na zrozumienie specyfiki symulacji lotów raketowych oraz algorytmów sterujących, co jest kluczowe dla skutecznego rozwoju systemów sterowania raketami. Dzięki zapoznaniu się z obecnymi rozwiązaniami możliwe było stworzenie własnego, dedykowanego narzędzia do symulacji lotów raketowych oraz algorytmu sterującego.

1.4.1 Silniki symulacji lotów raketowych.

Na rynku dostępnych jest wiele modeli silników fizycznych umożliwiających symulację lotów raketowych, jak i narzędzi umożliwiających tworzenie własnych symulacji. Poniżej zaprezentowane zostały najpopularniejsze z nich.

Narzędzia służące do ogólnego modelowania symulacji fizycznych:

- **Symulink.**

Bardzo zaawansowane narzędzie do modelowania i prowadzenia symulacji systemów dynamicznych. Symulink wchodzi w skład pakietu Matlab. System ten umożliwia elastyczne i dokładne modelowanie sił. Narzędzie to bazuje na krokowych modelach składania sił, co pozwala na bardzo dokładne modelowanie sił działających na obiekt. Symulink jest narzędziem płatnym oraz wymagającym dużej wiedzy z zakresu modelowania systemów dynamicznych.

Symulink może zostać wykorzystany do stworzenia bardzo dokładnego modelu symulacji lotu rakiety poprzez zamodelowanie sił działających na obiekt i symulację ich działania w czasie.

- **ANSYS Fluent.**

Zaawansowane narzędzie do symulacji dynamiki płynów (CFD – Computational Fluid Dynamics) wykorzystywane w inżynierii aerodynamicznej i mechanicznej. ANSYS Fluent umożliwia modelowanie przepływów płynów, wymiany ciepła, oraz oddziaływania sił aerodynamicznych na obiekty w różnych warunkach środowiskowych. ANSYS Fluent bazuje na rozwiązaniach równań Naviera-Stokesa, co pozwala na dokładne odwzorowanie zjawisk takich jak opór aerodynamiczny, turbulencje oraz efekty cieplne. Dzięki zaawansowanym algorytmom i funkcjom, Fluent umożliwia analizę złożonych scenariuszy, takich jak przepływ wokół wieloelementowych konstrukcji czy interakcje z gazami spalinowymi. Oprogramowanie to jest komercyjne, a jego obsługa wymaga wiedzy z zakresu CFD oraz doświadczenia w konfiguracji symulacji. ANSYS Fluent może być wykorzystany do szczegółowego modelowania lotu rakiety poprzez analizę przepływu powietrza wokół korpusu rakiety, co pozwala na optymalizację jej konstrukcji i poprawę stabilności aerodynamicznej.

- **OpenFOAM.**

Otwartoźródłowe oprogramowanie do symulacji CFD, które oferuje zaawansowane możliwości modelowania przepływów płynów, wymiany ciepła oraz innych procesów fizycznych. OpenFOAM opiera się na podejściu siatkowym (mesh-based), co pozwala na precyzyjne odwzorowanie geometrii i rozwiązywanie równań Naviera-Stokesa w ramach analiz aerodynamicznych. Główną zaletą OpenFOAM jest jego elastyczność i dostępność – oprogramowanie jest darmowe i można je dostosować do indywidualnych potrzeb poprzez tworzenie własnych modułów i skryptów. Jednak jego obsługa wymaga zaawansowanej wiedzy

technicznej oraz umiejętności pracy w środowisku tekstowym, co może być barierą dla początkujących użytkowników. OpenFOAM może zostać wykorzystany do symulacji lotu rakiety poprzez analizę sił aerodynamicznych oraz optymalizację kształtu i konstrukcji rakiety w celu minimalizacji oporu i poprawy wydajności lotu.

Dedykowane silniki fizyczne umożliwiające symulację lotów raketowych:

- **OpenRocket.**

Bezpłatne i otwartoźródłowe narzędzie stworzone specjalnie do symulacji lotów rakiet. OpenRocket jest łatwe w obsłudze, dzięki intuicyjnemu interfejsowi graficznemu, który umożliwia szybkie projektowanie rakiet oraz przeprowadzanie symulacji ich lotu. Program oferuje możliwość modelowania sił aerodynamicznych, takich jak opór powietrza, siła nośna oraz działanie silników raketowych w różnych warunkach. OpenRocket obsługuje różne typy silników, pozwala na analizę stabilności rakiety, trajektorii lotu oraz przewidywanie maksymalnej wysokości i zasięgu. Narzędzie to jest szczególnie popularne wśród hobbystów oraz studentów, dzięki swojej prostocie i dostępności, choć może mieć ograniczenia w przypadku bardziej zaawansowanych symulacji.

Narzędzie to było też w dużym stopniu inspiracją niniejszej pracy inżynierskiej.

- **RockSim.**

Komercyjne oprogramowanie do projektowania i symulacji lotów rakiet, dedykowane zarówno dla entuzjastów, jak i profesjonalistów. RockSim umożliwia projektowanie rakiet od podstaw, uwzględniając szczegółowe parametry takie jak geometria, masa komponentów czy rozmieszczenie środka ciężkości i ciśnienia. Program oferuje zaawansowane funkcje symulacji aerodynamicznych, takich jak analiza stabilności, przewidywanie trajektorii lotu oraz obliczanie wpływu warunków atmosferycznych (wiatr, temperatura) na zachowanie rakiety. Dzięki swojej wszechstronności, RockSim jest używany zarówno w edukacji, jak i w projektach zaawansowanych konstrukcji raketowych.

- **RAS Aero.**

RAS Aero to zaawansowane narzędzie do symulacji lotów rakiet dedykowane użytkownikom, którzy potrzebują dużej dokładności w analizie aerodynamicznej. Program oferuje możliwość modelowania trajektorii z uwzględnieniem szczegółowych parametrów aerodynamicznych, takich jak współczynniki oporu czy siły nośnej, oraz wpływu zmiennych warunków atmosferycznych na raketę. RAS Aero jest

szczególnie cenione za swoją zdolność do przewidywania zachowania rakiet przy dużych prędkościach oraz podczas przejścia przez bariery dźwiękowe. Oprogramowanie to jest płatne i wymaga większego doświadczenia w pracy z danymi aerodynamicznymi, ale oferuje bardzo dokładne wyniki symulacji.

1.4.2 Algorytmy uczenia maszynowego oparte o RL.

W dziedzinie algorytmów uczenia maszynowego opartych o Reinforcement Learning istnieje wiele narzędzi, jak i gotowych programów, które mogą być wykorzystane do uczenia modeli na symulacjach fizycznych.

Najpopularniejsze narzędzia stosowane do uczenia maszynowego RL w symulacjach fizycznych to m.in.:

- **TensorFlow.**

TensorFlow to popularna platforma open-source'owa do uczenia maszynowego, która znalazła szerokie zastosowanie w symulacjach fizycznych, w tym w dziedzinie reinforcement learning (RL). Dzięki wsparciu dla zaawansowanych algorytmów optymalizacji, TensorFlow umożliwia trenowanie modeli RL na danych pochodzących z symulacji fizycznych. Framework ten oferuje bogatą funkcjonalność do tworzenia i trenowania sieci neuronowych, a także integrację z innymi narzędziami do analizy danych i wizualizacji. TensorFlow wspiera różnorodne techniki optymalizacji, w tym metody oparte na gradientach, co pozwala na skuteczne rozwiązywanie problemów związanych z dynamiką fizyczną. Jego rozbudowana dokumentacja oraz wsparcie dla rozwoju modeli na dużą skalę czynią go jednym z najczęściej wykorzystywanych narzędzi w branży.

- **PyTorch.**

PyTorch to kolejny popularny framework open-source'owy, który jest szczególnie ceniony wśród badaczy i inżynierów zajmujących się uczeniem maszynowym. Dzięki swojej elastyczności oraz wsparciu dla dynamicznych grafów obliczeniowych, PyTorch doskonale nadaje się do tworzenia i trenowania modeli reinforcement learning w kontekście symulacji fizycznych. PyTorch jest często wykorzystywany w projektach związanych z robotyką oraz modelowaniem dynamicznych systemów, takich jak rakiety, ponieważ pozwala na łatwą integrację z fizycznymi symulacjami i oferuje szeroki zakres algorytmów RL. PyTorch charakteryzuje się także rozbudowaną społecznością oraz licznymi zasobami edukacyjnymi, co czyni go atrakcyjnym wyborem dla osób rozpoczynających pracę z uczeniem maszynowym.

- **Stable Baselines3.**

Stable Baselines3 to biblioteka open-source'owa zbudowana na bazie PyTorch, która oferuje gotowe implementacje popularnych algorytmów reinforcement learning, takich jak PPO (Proximal Policy Optimization), A2C (Advantage Actor-Critic) czy DQN (Deep Q-Network). Jest to narzędzie szczególnie przydatne w kontekście symulacji fizycznych, gdzie wymagana jest szybka i efektywna implementacja algorytmów RL do testowania i optymalizacji systemów sterowania. Stable Baselines3 jest dobrze udokumentowaną platformą, która ułatwia implementację i eksperymentowanie z różnymi metodami RL. Dzięki swojej prostocie i kompatybilności z popularnymi symulatorami, jak Gazebo czy Unity, biblioteka ta stała się jednym z najczęściej wykorzystywanych narzędzi w projektach związanych z uczeniem maszynowym i symulacjami fizycznymi.

Poniżej przedstawione zostały jedne z popularniejszych oraz częściej wykorzystywanych narzędzi służących do trenowania modeli sztucznej inteligencji sterujących lotami rakiet.

- **Simulink z Aerospace Blockset.**

MATLAB w połączeniu z Simulinkiem oraz dodatkiem Aerospace Blockset to zaawansowane narzędzie umożliwiające symulację lotu rakiet. Oprogramowanie to pozwala na elastyczne modelowanie dynamiki obiektów latających, uwzględniając siły aerodynamiczne, grawitację i inne istotne parametry fizyczne. Aerospace Blockset oferuje gotowe bloki do analizy trajektorii oraz projektowania systemów sterowania. Dodatkowo, MATLAB umożliwia integrację z algorytmami uczenia maszynowego, co pozwala na trenowanie modeli sterowania w oparciu o dane z symulacji. System ten jest płatny i wymaga zaawansowanej wiedzy z zakresu modelowania dynamicznego, jednak oferuje bardzo wysoką precyzję i wsparcie dla projektów naukowych i inżynierskich.

- **Microsoft AirSim.**

Microsoft AirSim to open-source'owy symulator lotu, pierwotnie zaprojektowany dla dronów, ale możliwy do dostosowania do symulacji rakiet. Oprogramowanie to obsługuje realistyczne warunki fizyczne, takie jak zmienne aerodynamiczne i różne środowiska lotu. AirSim integruje się z popularnymi frameworkami uczenia maszynowego, takimi jak TensorFlow czy PyTorch, umożliwiając trenowanie modeli sterowania w oparciu o reinforcement learning. Dzięki elastyczności i

wsparciu dla programowania API, użytkownik może tworzyć niestandardowe środowiska do symulacji. AirSim jest darmowy, ale jego zastosowanie wymaga dostosowania środowiska do specyficznych potrzeb użytkownika, co może być czasochłonne.

- **Gazebo + ROS.**

Gazebo to open-source'owy symulator fizyki, który w połączeniu z systemem ROS (Robot Operating System) stanowi potężne narzędzie do modelowania i sterowania obiektami dynamicznymi, takimi jak rakiety. Gazebo oferuje zaawansowaną symulację fizyczną, w tym modelowanie sił aerodynamicznych, kolizji i zmiennych środowiskowych. Integracja z ROS umożliwia tworzenie i implementację algorytmów sterowania opartych na uczeniu maszynowym, takich jak reinforcement learning. Platforma wspiera również analizę danych w czasie rzeczywistym, co czyni ją idealnym narzędziem do testowania i optymalizacji systemów sterowania. Gazebo + ROS jest darmowe, ale wymaga zaawansowanej konfiguracji oraz wiedzy z zakresu programowania i modelowania fizycznego.

1.4.3 Podsumowanie.

Jak zostało to przedstawione w tym rozdziale, pomimo dość niszowego zagadnienia, na rynku dostępnych jest wiele narzędzi, które mogą być wykorzystane do stworzenia symulacji lotu rakiety oraz trenowania modeli sztucznej inteligencji. Narzędzia te są w większości bardzo zaawansowane i wymagają dużego doświadczenia oraz wiedzy w dziedzinie symulacji, fizyki, jak i uczenia maszynowego. Często są to także narzędzia płatne. Niewątpliwie jednak rozwiązania te oferują symulacje o dużej dokładności i szczegółowości, dzięki zastosowaniu skomplikowanych rozwiązań takich jak CFD czy analiza trajektorii lotu. Użycie dokładnych symulacji i trenowanie modeli, za pomocą dobrze dobranych algorytmów umożliwia osiągnięcie bardzo dobrych wyników w sterowaniu lotem rakiety.

W niniejszej pracy inżynierskiej wykorzystane zostały własne implementacje silnika fizycznego oraz algorytmu sterującego, co pozwoliło na pełną kontrolę nad procesem tworzenia i testowania systemu sterującego rakieta. Efektem tego jest także uproszczony model fizyczny, o czym mowa jest w kolejnych rozdziałach pracy. Rozwiązanie prezentowane w tej pracy odbiega od profesjonalnych narzędzi pod względem skomplikowania i dokładności, jednak pozwala na zrozumienie sposobu działania symulacji lotu rakiety

oraz algorytmów sterujących i jest dobrą bazą do rozwoju bardziej zaawansowanych systemów w przyszłości.

2 Moduł I: Silnik fizyczny symulacji lotów rakietowych.

Silnik fizyczny do symulacji lotów rakietowych to pierwszy z modułów niniejszej pracy inżynierskiej. Za pomocą tego projektu osiągnięty zostaje cel pośredni pracy - stworzenie środowiska do uczenia algorytmów. Autorem tego modułu jest Jakub Kindlik.

W tym rozdziale przedstawione zostały teoretyczne podstawy fizyczne, użyte technologie, oraz trzy główne komponenty tego modułu:

- Projekt silnika fizycznego.
- Model sił działających na raketę.
- Moduł wizualizacyjny symulacji.

Omówione zostały także zagadnienia związane ze sprawdzeniem dokładności symulacji oraz ewolucją projektu.

2.1 Założenia projektowe.

W trakcie pracy nad modułem silnika fizycznego symulacji lotów rakietowych poczyniono szereg założeń, które miały na celu ułatwienie implementacji oraz zapewnienie odpowiedniej wydajności symulacji. Poniżej przedstawiono najważniejsze z nich:

- **Modelowanie uproszczonych sił działających na raketę.**

W celu zmniejszenia złożoności obliczeniowej model sił działających na raketę zostanie odpowiednio uproszczony. Uwzględnione zostaną jedynie kluczowe siły, takie jak grawitacja, opór powietrza i siła ciągu silnika. Takie podejście umożliwia łatwiejsze zamodelowanie systemu i redukuje zapotrzebowanie na zasoby obliczeniowe.

- **Optymalizacja pod kątem pracy na komputerze osobistym.**

Symulacje będą wykonywane na komputerze osobistym, dlatego silnik fizyczny musi być zoptymalizowany pod kątem wydajności, jak i okrojony w kwestii skomplikowanych obliczeń. Ograniczenie zasobożerności jest konieczne, aby

zapewnić płynne działanie nawet na sprzęcie o przeciętnych parametrach.

- **Uproszczona geometria rakiety.**

W celu ograniczenia złożoności modelu fizycznego rakietę będzie miała kształt walca i zostanie pozbawiona skrzydeł. Taka decyzja pozwala na skupienie się na kluczowych aspektach dynamiki lotu bez nadmiernego komplikowania obliczeń.

- **Sterowanie poprzez obracany silnik.**

Układ sterujący rakieta będzie się opierał na możliwości obracania silnika, co bezpośrednio wpłynie na trajektorię lotu. Taki model jest prosty do zamodelowania, a jednocześnie pozwala na sterowanie rakieta przez algorytmy samouczące, poprzez zmianę jednego parametru.

- **Możliwość modyfikacji parametrów symulacji w czasie rzeczywistym.**

Symulacja zostanie zaprojektowana w sposób umożliwiający ingerencję w jej parametry w trakcie trwania. W szczególności modyfikowanym parametrem powinien być kąt nachylenia silnika. Taka funkcjonalność pozwala na uczenie algorytmów sterujących w czasie rzeczywistym.

- **Możliwość wizualizacji symulacji.**

Symulacja będzie wyposażona w opcjonalną funkcję wizualizacji, co umożliwi analizę trajektorii i zachowania rakiety w sposób intuicyjny. Jednak wizualizacja nie będzie wymagana, co pozwoli ograniczyć zużycie zasobów obliczeniowych w trakcie symulacji.

Założenia te konieczne są do stworzenia silnika fizycznego, który będzie w stanie współpracować z modułem algorytmu sterującego rakieta, jednocześnie będąc na tyle prostym, aby nie obciążać zasobów komputera osobistego.

2.2 Teoretyczne podstawy fizyczne.

Tworzenie silnika symulacji lotów raketowych wymaga zrozumienia zasad fizyki odnoszących się do ruchu rakiety w atmosferze, a także zjawisk towarzyszących temu ruchowi. Ponadto konieczne jest zrozumienie podstawowych zagadnień z zakresu matematyki, w tym geometrii przestrzennej oraz równań różniczkowych.

W tym rozdziale po krótko omówione zostały zagadnienia, które stanowią podstawę implementacji silnika fizycznego symulacji lotów raketowych.

2.2.1 Opis sił działających na raketę

W trakcie lotu rakiety w atmosferze działają na nią różne siły, które determinują zarówno ruch postępowy, jak i obrotowy.

Najważniejszymi z tych sił są:

- **Siła ciężkości (Gravity)**

Siła skierowana ku środkowi Ziemi, działająca na środek masy rakiety (*Center of Gravity, CoG*). Jej wartość wyraża się jako:

$$\vec{F}_g = m \cdot \vec{g},$$

gdzie:

- m – masa rakiety [kg],
- $\vec{g}$ – wektor przyspieszenia grawitacyjnego [m/s²], zazwyczaj o wartości około 9.81 m/s² na poziomie morza.

- **Siła nośna (Lift)**

Siła działająca na punkt środka parcia (*Center of Pressure, CoP*) rakiety, prostopadła do kierunku przepływu powietrza. Jest istotna w stabilizacji lotu i zależy od kształtu rakiety oraz kąta natarcia. Wyraża się jako:

$$\vec{F}_l = \frac{1}{2} \rho v^2 C_l A,$$

gdzie:

- ρ – gęstość powietrza [kg/m³],
- v – prędkość rakiety względem powietrza [m/s],
- C_l – współczynnik siły nośnej (zależy od kąta natarcia),
- A – efektywna powierzchnia rakiety [m²].

- **Siła oporu aerodynamicznego (Drag)**

Siła działająca również na punkt *Center of Pressure (CoP)*, przeciwna do kierunku ruchu. Zależy od właściwości powietrza i kształtu rakiety, a jej wartość wyraża się jako:

$$\vec{F}_d = \frac{1}{2} \rho v^2 C_d A,$$

gdzie:

- C_d – współczynnik oporu aerodynamicznego (zależy od kształtu rakiety).

- **Siła ciągu (Thrust)**

Siła generowana przez silnik rakiety, działająca na punkt *Center of Thrust* (*CoT*) wzdłuż osi rakiety. Wyrażenie opisujące siłę ciągu to:

$$\vec{F}_t = F_{max} \cdot u(t),$$

gdzie:

- F_{max} – maksymalna siła ciągu [N],
- $u(t)$ – funkcja sterująca (np. zależna od przepływu paliwa).

2.2.2 Równania ruchu postępowego

Ruch postępowy rakiety w atmosferze opisuje druga zasada dynamiki Newtona, zgodnie z którą suma wszystkich sił działających na raketę równa jest iloczynowi masy rakiety i przyspieszenia środka masy:

$$\vec{F}_{net} = \vec{F}_g + \vec{F}_l + \vec{F}_d + \vec{F}_t.$$

Przyspieszenie rakiety można więc wyznaczyć jako:

$$\vec{a} = \frac{\vec{F}_{net}}{m},$$

natomiast prędkość w kolejnych momentach czasu oblicza się poprzez całkowanie przyspieszenia względem czasu:

$$\vec{v}(t) = \vec{v}_0 + \int_0^t \vec{a}(\tau) d\tau,$$

gdzie:

- $\vec{v}_0$ – początkowa prędkość rakiety [m/s].

Pozycję rakiety można wyznaczyć poprzez kolejne całkowanie prędkości:

$$\vec{x}(t) = \vec{x}_0 + \int_0^t \vec{v}(\tau) d\tau,$$

gdzie:

- $\vec{x}_0$ – początkowa pozycja rakiety [m].

2.2.3 Ruch obrotowy i momenty sił

Ruch obrotowy rakiety jest determinowany momentami sił działającymi na raketę. Kluczowe znaczenie ma tutaj punkt środka masy (CoG), wokół którego rakieta może rotować. Moment siły względem środka masy wyraża się jako:

$$\vec{M} = \vec{r} \times \vec{F},$$

gdzie:

- $\vec{r}$ – wektor od środka masy do punktu przyłożenia siły [m],
- $\vec{F}$ – siła działająca na raketę [N].

Równania ruchu obrotowego

Moment obrotowy generuje przyspieszenie kątowe zgodnie z równaniem:

$$\vec{\alpha} = \frac{\vec{M}}{I},$$

gdzie:

- $\vec{\alpha}$ – przyspieszenie kątowe [rad/s²],
- I – moment bezwładności rakiety względem osi obrotu [kg·m²].

Następnie prędkość kątowna rakiety $\vec{\omega}$ zmienia się w czasie według:

$$\vec{\omega}(t) = \vec{\omega}_0 + \int_0^t \vec{\alpha}(\tau) d\tau,$$

gdzie:

- $\vec{\omega}_0$ – początkowa prędkość kątowna [rad/s].

Kąt orientacji rakiety w przestrzeni zmienia się w czasie:

$$\vec{\theta}(t) = \vec{\theta}_0 + \int_0^t \vec{\omega}(\tau) d\tau,$$

gdzie:

- $\vec{\theta}_0$ – początkowy kąt orientacji [rad].

Stabilizacja lotu

Ruch obrotowy jest ściśle związany ze stabilnością rakiety w locie. Rakieta powinna być tak zaprojektowana, aby środek parcia (CoP) znajdował się poniżej środka masy (CoG). Dzięki temu moment siły aerodynamicznej działa stabilizująco, przeciwdziałając niekontrolowanym obrotom.

2.2.4 Modelowanie środowiska atmosferycznego

Środowisko atmosferyczne wpływa na dynamikę ruchu rakiety, szczególnie poprzez gęstość powietrza (ρ) oraz zmienne ciśnienie i temperaturę. Modelowanie środowiska wymaga uwzględnienia:

- Zmiennej gęstości powietrza w zależności od wysokości:

$$\rho(h) = \rho_0 \cdot \exp\left(-\frac{h}{H}\right),$$

gdzie:

- ρ_0 – gęstość powietrza na poziomie morza [kg/m^3],
 - h – wysokość [m],
 - H – charakterystyczna wysokość skali atmosfery [m].
- Efektów zmian temperatury i ciśnienia w różnych warstwach atmosfery.

2.2.5 Kąt natarcia rakiety

Kąt natarcia (α) jest kluczowym parametrem aerodynamicznym opisującym orientację rakiety względem przepływu powietrza. Określa on, pod jakim kątem oś symetrii rakiety jest nachylona do wektora prędkości względnej powietrza.

Definicja kąta natarcia

Kąt natarcia α definiuje się jako kąt pomiędzy wektorem prędkości powietrza $\vec{v}_{rel}$ a osią symetrii rakiety. Wyrażenie to zapisuje się jako:

$$\alpha = \arccos\left(\frac{\vec{v}_{rel} \cdot \vec{a}}{\|\vec{v}_{rel}\| \|\vec{a}\|}\right),$$

gdzie:

- $\vec{v}_{rel}$ – wektor prędkości względnej przepływu powietrza,

- $\vec{a}$ – wektor osi symetrii rakiety,
- $\|\vec{v}_{rel}\|$ – wartość prędkości względnej [m/s],
- $\|\vec{a}\|$ – wartość wektora osi symetrii (zwykle 1 jako wektor jednostkowy).

Wpływ kąta natarcia na dynamikę rakiety

Kąt natarcia ma istotny wpływ na siły aerodynamiczne, w szczególności:

- **Siła nośna** ($\vec{F}_l$): Przy większych kątach natarcia wzrasta siła nośna, która stabilizuje raketę, ale również zwiększa ryzyko niestabilności aerodynamicznej.
- **Siła oporu** ($\vec{F}_d$): Przy większych kątach natarcia wzrasta siła oporu, co prowadzi do większych strat energetycznych w trakcie lotu.

Zakres kąta natarcia w rakietach

Dla ракет stabilnych aerodynamicznie kąt natarcia zazwyczaj jest mały (rzędu kilku stopni), aby zminimalizować siły destabilizujące. W czasie lotu kąt natarcia zmienia się dynamicznie w zależności od:

- warunków atmosferycznych,
- trajektorii lotu,
- działania układów sterujących.

2.2.6 Geometria rakiety

Opis geometrii rakiety jako walca

Rakieta została zamodelowana jako bryła o kształcie walca ze stożkową głowicą.

Taka uproszczona geometria pozwala na efektywne obliczenia przy zachowaniu wystarczającej dokładności w kontekście modelowania sił aerodynamicznych i dynamiki ruchu.

Wzory geometryczne

Dla uproszczonego modelu rakiety jako walca o promieniu r i wysokości h podstawowe właściwości geometryczne wyznacza się następująco:

- **Objętość walca:**

$$V_{cyl} = \pi r^2 h,$$

gdzie:

- r – promień podstawy walca [m],

– h – wysokość walca [m].

- **Powierzchnia boczna walca:**

$$A_{cyl} = 2\pi rh.$$

Powierzchnia przekroju rakiety jako walca pod różnymi kątami

Powierzchnia przekroju rakiety, modelowanej jako walec o promieniu r i wysokości h , zależy od kąta natarcia α względem przepływu powietrza.

Przekrój czołowy ($\alpha = 0^\circ$): Jeżeli rakieta porusza się wzdłuż swojej osi symetrii, przekrój czołowy jest równy powierzchni podstawy cylindra:

$$A_0 = \pi r^2.$$

Przekrój pod kątem α : Jeżeli rakieta jest nachylona pod kątem α do kierunku przepływu, powierzchnia efektywna przekroju zmienia się zgodnie z rzutem walca na płaszczyznę prostopadłą do przepływu. Powierzchnię tę można wyznaczyć jako:

$$A_\alpha = \pi r^2 \cos \alpha + 2rh \sin \alpha,$$

gdzie:

- $\cos \alpha$ odpowiada zmniejszeniu widocznej powierzchni podstawy wzdłuż osi symetrii,
- $2rh \sin \alpha$ dodaje widoczną powierzchnię boczną w rzucie.

Granice powierzchni przekroju:

- Dla $\alpha = 0^\circ$: $A_\alpha = A_0 = \pi r^2$ (przekrój czołowy).
- Dla $\alpha = 90^\circ$: $A_\alpha = 2rh$ (przekrój boczny).

Wartość A_α zależy więc silnie od kąta natarcia α , co wpływa na siły aerodynamiczne działające na raketę w locie.

Charakterystyka punktów charakterystycznych rakiety

W rakiecie wyróżnia się trzy kluczowe punkty geometryczne i dynamiczne, na które oddziałują wszelkie siły działające na lecącą raketę:

- **Środek masy (CoG):** Punkt, w którym skupiona jest całkowita masa rakiety. Dla symetrii osiowej i jednorodnej gęstości materiałów jest zlokalizowany na osi rakiety. Położenie CoG zmienia się w trakcie lotu w zależności od spalania paliwa i zmiany masy, choć w symulacji zakłada się zazwyczaj stałe położenie CoG.
- **Środek parcia (CoP):** Punkt, w którym można przyłożyć wypadkową siłę aerodynamiczną działającą na raketę. Położenie CoP zależy od kształtu rakiety i rozkładu ciśnienia aerodynamicznego.
- **Środek ciągu (CoT):** Punkt, w którym działa wypadkowa siła ciągu generowana przez silniki rakiety. Położenie CoT zależy od konfiguracji silników i ich rozmieszczenia.

Relacje między punktami Dla zapewnienia stabilności lotu rakiety:

- CoP powinien znajdować się poniżej CoG, co gwarantuje stabilność aerodynamiczną.
- CoT powinien być blisko osi rakiety, aby zminimalizować momenty obrotowe powodujące niestabilności.

2.2.7 Obroty układów współrzędnych

Układy współrzędnych w dynamice rakiety

Aby w dokładny sposób określać pozycje i obrót rakiety w przestrzeni konieczne jest zamodelowanie układów współrzędnych związanych z ruchem rakiety. W analizach ruchu rakiety wyróżnia się trzy podstawowe układy współrzędnych:

- **Układ inercjalny (Ziemia):** Stały układ odniesienia związany z Ziemią, w którym mierzone są położenie i orientacja rakiety. Oś z jest skierowana w stronę przeciwną do środka Ziemi.
- **Układ ciała (rakiet):** Układ poruszający się wraz z rakieta. Oś x wskazuje kierunek przodu rakiety, y jest prostopadła do osi x w płaszczyźnie skrzydeł (jeśli istnieją), a z jest skierowana w górę, w kierunku czuba rakiety.
- **Układ lokalny (silnik):** Układ związany z kierunkiem działania siły ciągu. Oś z wskazuje wektor ciągu, a osie y i x są prostopadłe do z .

Kąty Eulera i transformacje układów współrzędnych

Do opisu orientacji rakiety w przestrzeni wykorzystuje się kąty Eulera, umożliwiające transformacje układów współrzędnych za pomocą operacji macierzowych. Kąty Eulera

to trzy kąty opisujące orientację rakiety względem trzech osi układu inercyjnego:

- **Yaw** (ψ): Obrót wokół osi z w układzie Ziemi.
- **Pitch** (θ): Obrót wokół osi y w układzie Ziemi.
- **Roll** (ϕ): Obrót wokół osi x w układzie Ziemi.

Macierze rotacji Transformacja między układami współrzędnych odbywa się za pomocą macierzy rotacji. Dla kątów Eulera macierz rotacji R jest wynikiem trzech obrotów:

$$R = R_z(\psi)R_y(\theta)R_x(\phi),$$

gdzie:

$$R_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{bmatrix},$$

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix},$$

$$R_z(\psi) = \begin{bmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Dzięki kątom Eulera i macierzom rotacji możliwa jest transformacja wektorów i równań między różnymi układami współrzędnych, co jest kluczowe dla analizy trajektorii lotu rakiety oraz sterowania jej orientacją w przestrzeni.

2.2.8 Podsumowanie

Przedstawione równania i modele opisują w ogólny sposób dynamikę ruchu rakiety w atmosferze, uwzględniając zarówno ruch postępowy, jak i obrotowy oraz wpływ środowiska atmosferycznego. Poprawne zrozumienie i modelowanie tych elementów jest kluczowe dla skutecznej implementacji silnika symulacji fizycznych lotów raketowych.

2.3 Użyte technologie.

Do stworzenia silnika fizycznego symulacji lotów raketowych wykorzystane zostały technologie, które zostały opisane w rozdziale 2.3, gdzie przedstawiono technologie

zastosowane w obu modułach pracy inżynierskiej.

Dodatkowo do stworzenia modułu I użyte zostały poniższe biblioteki:

- **Biblioteki do analizy danych i obliczeń numerycznych:**

- **numpy**, wersja: 1.26.4,

Podstawowa biblioteka do obliczeń numerycznych w Pythonie. Oferuje wsparcie dla wielowymiarowych tablic oraz operacji na nich.

- **pandas**, wersja: 2.2.3,

Biblioteka do analizy danych, która ułatwia manipulowanie i przetwarzanie danych tabelarycznych. Znajduje szerokie zastosowanie w analizie danych i uczeniu maszynowym.

- **trimesh**, wersja: 4.5.3,

Biblioteka do manipulacji i analizy danych 3D, zwłaszcza trójwymiarowych siatek. Używana w aplikacjach CAD i analizie przestrzennej.

- **Biblioteki do pracy z GUI (interfejsami graficznymi):**

- **PyQt5**, wersja: 5.15.11,

Biblioteka do tworzenia interfejsów graficznych opartych na Qt. Umożliwia tworzenie aplikacji z bogatym GUI, wspiera wiele platform.

- **PyQt5-Qt5**, wersja: 5.15.2,

Zestaw narzędzi i komponentów Qt5 dla PyQt5, pozwalający na korzystanie z funkcji Qt5 w aplikacjach Python. Używana do tworzenia profesjonalnych aplikacji graficznych.

- **QtPy**, wersja: 2.4.2,

Zestaw narzędzi do pracy z Qt w Pythonie, umożliwiający stosowanie różnych wersji Qt w jednym projekcie. Używany do tworzenia aplikacji graficznych.

- **Biblioteki do grafiki i renderowania 3D:**

- **PyOpenGL**, wersja: 3.1.7,

Biblioteka umożliwiająca tworzenie grafiki 3D przy użyciu OpenGL w Pythonie. Wykorzystywana w aplikacjach, które wymagają renderowania grafiki komputerowej.

- **PyOpenGL-accelerate**, wersja: 3.1.7,

Przyspieszenie obliczeń OpenGL w Pythonie poprzez implementację krytycznych operacji w C. Stosowana w aplikacjach wymagających wysokiej wydajności

w grafice 3D.

- **pyqtgraph**, wersja: 0.13.7,

Biblioteka do tworzenia dynamicznych wizualizacji danych i wykresów w czasie rzeczywistym. Znajduje zastosowanie w aplikacjach naukowych i inżynierskich.

- **Biblioteki do testowania i rozwoju oprogramowania:**

- **pytest**, wersja: 8.3.3,

Popularne narzędzie do testowania w Pythonie, umożliwiające łatwe pisanie testów jednostkowych. Używane w procesie ciągłej integracji i weryfikacji kodu.

- **pluggy**, wersja: 1.5.0,

Biblioteka do tworzenia i zarządzania punktami rozszerzeń w aplikacjach. Jest szeroko wykorzystywana w narzędziach testowych, takich jak pytest.

- **python-utils**, wersja: 3.9.1,

Zbiór różnych narzędzi i funkcji pomocniczych dla Pythona. Zawiera funkcje do obsługi typów danych, plików i wyjątków.

- **Wiele innych mniej istotnych bibliotek.**

Wymienione powyżej biblioteki są głównymi narzędziami, z których korzystano podczas tworzenia modułu I pracy inżynierskiej. Oprócz nich użyto także wielu innych bibliotek pomocniczych, które ułatwiły pracę nad projektem. Są to między innymi biblioteki do obsługi plików, pracy z pakietami czy zarządzania danymi czasowymi.

2.4 Projekt silnika fizycznego.

Silnik fizyczny do symulacji lotów raketowych będący głównym komponentem modułu I niniejszej pracy inżynierskiej został zaprojektowany zgodnie z założeniami opisanymi w rozdziale 2.1. W niniejszym rozdziale przedstawiono szczegóły architektury i implementacji silnika fizycznego.

2.4.1 Architektura systemu symulacji

System symulacji został zaprojektowany z wykorzystaniem paradygmatu programowania obiektowego, co pozwala na modularność i łatwą rozbudowę silnika o wykorzystanie

nowych sił. Architektura systemu oparta jest na dobrze zdefiniowanych interfejsach i klasach, co przedstawiono na rysunku 2.1.

W dalszej części tego rozdziału szczegółowo opisane zostały poszczególne elementy architektury systemu symulacji.

2.4.2 Interfejsy

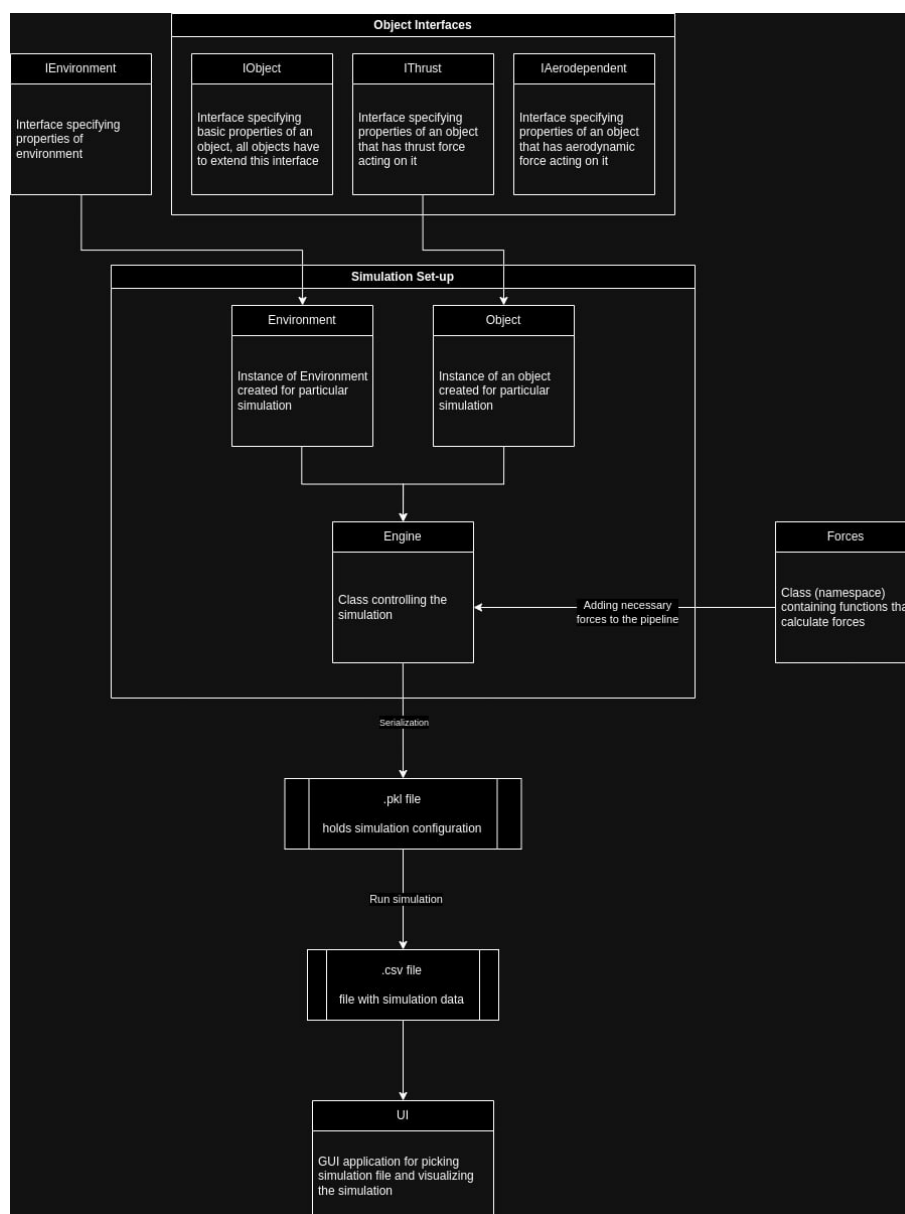
Interfejsy definiują zbiór metod i atrybutów, które muszą być zaimplementowane przez klasy, które je dziedziczą. Dzięki nim obiekty, które powinny posiadać określone właściwości, mogą być traktowane jednolicie, co ułatwia zarządzanie nimi i zapewnia spójność w systemie. Aby obiekt mógł korzystać z konkretnego rodzaju sił, musi implementować odpowiednie interfejsy, które to umożliwiają. Dzięki temu, w łatwy sposób można rozszerzyć dany obiekt o parametry konieczne do symulacji nowych sił. Pozwala to na elastyczność silnika i jego łatwe dostosowanie do różnych wersji symulacji.

W stworzonym na potrzeby tej pracy silniku fizycznym wyróżniamy następujące interfejsy:

‘IObject’

Interfejs ‘IObject’ jest podstawowym interfejsem dla wszystkich obiektów w symulacji. Przez obiekty rozumie się rakiety, na które oddziałują siły podczas symulacji. Interfejs ten definiuje podstawowe właściwości rakiety, które powinna ona posiadać bez względu na to, jakie siły będą ostatecznie na nią oddziaływały. W obecnej implementacji rakietę traktowana jest jako walec o jednolitej dystrybucji masy.

```
1 class IObject(ABC):
2     def __init__(self, name: str, object_model_path: str, height:
      float, mass: float = 1,
3         radius: float = 0.1, position:
          ndarray[ndarray.float64] = None, rotation:
          ndarray[ndarray.float64] = None, velocity:
          ndarray[ndarray.float64] = None,
4         acceleration: ndarray[ndarray.float64] = None,
          angular_velocity: ndarray[ndarray.float64] = None,
          angular_acceleration: ndarray[ndarray.float64] =
          None):
5
6         self.config = Config()
```



Rysunek 2.1. Architektura systemu symulacji

```

7         self.name = name
8         self.position = position
9         self.velocity = velocity
10        self.acceleration = acceleration
11        self.angular_velocity = angular_velocity
12        self.angular_acceleration = angular_acceleration
13        self.rotation = rotation
14        self.mass = mass
15        self._height = height
16        self._radius = radius
17        self._inertia_tensor = self.calculate_inertia_tensor()
18        self.object_model = self.load_model(object_model_path)

```

W konstruktorze klasy implementującej ten interfejs inicjalizowane są następujące parametry:

- 'name': Nazwa obiektu.
- 'object_model_path': Ścieżka do modelu 3D obiektu.
- 'height': Wysokość obiektu.
- 'mass': Masa obiektu (domyślnie 1 kg).
- 'radius': Promień obiektu (domyślnie 0.1 m).
- 'position': Pozycja obiektu w przestrzeni (domyślnie 'None').
- 'velocity': Prędkość obiektu (domyślnie 'None').
- 'acceleration': Przyspieszenie obiektu (domyślnie 'None').
- 'angular_velocity': Kątowa prędkość obrotu obiektu (domyślnie 'None').
- 'angular_acceleration': Przyspieszenie kątowe obiektu (domyślnie 'None').

Dodatkowo, konstruktor inicjalizuje:

- 'self.config': Konfiguracja obiektu.
- 'self._inertia_tensor': Tensor bezwładności obiektu.
- 'self.object_model': Model obiektu 3D.

Metody w interfejsie IObject:

- 'load_model(object_model_path)': Ładuje model 3D obiektu z podanej ścieżki. Model jest skalowany i translokowany do odpowiednich wymiarów.
- 'calculate_inertia_tensor()': Oblicza tensor bezwładności obiektu na podstawie jego masy i wymiarów.
- 'update(position=None, rotation=None, velocity=None)': Uaktualnia parametry

obiektu: pozycję, rotację i prędkość.

- `'get_data()'`: Zwraca dane obiektu w formie tekstowej, zawierającą informacje o pozycji, rotacji, prędkości, przyspieszeniu.
- `'__str__()'`: Zwraca reprezentację obiektu w formie tekstowej (pozycja, rotacja, prędkość, przyspieszenie).

'IThrust'

Interfejs `'IThrust'` rozszerza interfejs `'IObject'` o właściwości specyficzne dla obiektów posiadających silnik. Każdy obiekt, implementujący ten interfejs posiada wszystkie własności konieczne do obliczania sił ciągu, nań działających.

```
1 class IThrust(ABC):
2     def __init__(self, cot, engine_angle: NDArray[np.float64] =
3         None):
4         self._cot = cot
5         self.engine_angle = engine_angle
6
7         if not isinstance(self, IObject):
8             raise RuntimeError("Object needs to extend IObject to
9                 be IThrust")
```

W konstruktorze deklarowane są następujące parametry:

- `'cot'`: Odległość środka ciągu (Center of Thrust) w osi Z od środka masy rakiety.
- `'engine_angle'`: Kąt nachylenia silnika względem lokalnego układu współrzędnych rakiety.

Jeśli obiekt nie rozszerza klasy `'IObject'`, zgłaszany jest błąd.

Opis metod:

- `'mass_flow_rate(time: float)'`: Metoda abstrakcyjna, która oblicza współczynnik przepływu masy w danym czasie.
- `'exhaust_velocity(time: float)'`: Metoda abstrakcyjna, która oblicza prędkość wylotową spalin w danym czasie.

'IAerodependent'

Interfejs `'IAerodependent'` rozszerza interfejs `'IObject'` o właściwości związane z oddziaływaniem aerodynamicznym, takie jak powierzchnia czołowa, współczynnik oporu i inne. Dzięki implementacji tego interfejsu, obiekt posiada wszystkie własności

i parametry pozwalające na obliczanie sił aerodynamicznych działających na ten obiekt.

Poniżej przedstawiamy kod tego interfejsu:

```
1 class IAerodependent(ABC):
2     def __init__(self, cop, angle_of_attack: NDArray[np.float64] =
      None, relative_velocity: NDArray[np.float64] = None,
      drag_coefficient: float =
      C.DRAG_COEFFICIENT_PARALLEL.value):
3         self._cop = cop
4         self.angle_of_attack = angle_of_attack
5         self.relative_velocity = relative_velocity
6         self._reference_area = self.calculate_reference_area()
7         self.drag_coefficient = drag_coefficient
8
9         if not isinstance(self, IObject):
10             raise RuntimeError("Object needs to extend IObject to
              be IAerodependent")
11
12     def calculate_reference_area(self: IObject):
13         reference_area = 0
14
15         for face in self.object_model.faces:
16             vertices = self.object_model.vertices[face]
17             projected_vertices = vertices[:, :2]
18
19             x = projected_vertices[:, 0]
20             y = projected_vertices[:, 1]
21
22             area = 0.5 * np.abs(np.dot(x, np.roll(y, 1)) -
              np.dot(y, np.roll(x, 1)))
23
24             reference_area += area
25
26         return reference_area
```

W konstruktorze deklarowane są następujące wartości parametrów:

- ‘cop’: Odległość środka ciśnienia (Center of Pressure) w osi Z od środka masy rakiety.
- ‘angle_of_attack’: Kąt natarcia rakiety względem kierunku przepływu powietrza.
- ‘relative_velocity’: Względna prędkość rakiety względem przepływającego powietrza.

- ‘drag_coefficient’: Współczynnik oporu (domyślnie wartość zdefiniowana w konfiguracji).

Opis metod:

- ‘calculate_reference_area()’: Oblicza powierzchnię czołową obiektu na podstawie modelu 3D.

‘IEnvironment’

Interfejs ‘IEnvironment’ definiuje właściwości środowiska symulacji, takie jak gęstość powietrza, prędkość dźwięku, temperatura, ciśnienie, wiatr i gradient wiatru.

```

1 from abc import ABC, abstractmethod
2
3 class IEnvironment(ABC):
4     @abstractmethod
5     def get_air_density(self, position=None) -> float:
6         raise NotImplementedError
7
8     @abstractmethod
9     def get_speed_of_sound(self, position=None) -> float:
10        raise NotImplementedError
11
12    @abstractmethod
13    def get_temperature(self, position=None) -> float:
14        raise NotImplementedError
15
16    @abstractmethod
17    def get_pressure(self, position=None) -> float:
18        raise NotImplementedError
19
20    @abstractmethod
21    def get_wind(self, position=None) -> list:
22        raise NotImplementedError
23
24    @abstractmethod
25    def get_wind_gradient(self, position=None) -> list:
26        raise NotImplementedError

```

Opis metod:

- ‘get_air_density(position=None)’: Metoda, która zwraca gęstość powietrza w zadanej pozycji. Jeśli ‘position’ nie zostanie podane, metoda użyje domyślnej

wartości.

- `'get_speed_of_sound(position=None)'`: Metoda, która zwraca prędkość dźwięku w zadanej pozycji.
- `'get_temperature(position=None)'`: Metoda, która zwraca temperaturę w zadanej pozycji.
- `'get_pressure(position=None)'`: Metoda, która zwraca ciśnienie w zadanej pozycji.
- `'get_wind(position=None)'`: Metoda, która zwraca listę informacji o wietrze w danej pozycji.
- `'get_wind_gradient(position=None)'`: Metoda, która zwraca gradient wiatru w danej pozycji.

2.4.3 Klasy

Klasy w silniku fizycznym odpowiadają za modelowanie sił oraz zapewniają działanie symulacji fizycznej. Główne klasy, które pełnią kluczowe role w systemie, to:

- **'PhysicsEngine'**: Jest to najważniejsza klasa silnika, odpowiedzialna za zarządzanie przebiegiem symulacji. Oblicza siły działające na obiekty, aktualizuje ich pozycje, prędkości oraz momenty obrotowe, a także zapisuje wyniki do plików.
- **'Forces'**: Klasa ta zawiera różne typy sił działających na obiekty w symulacji. W ramach tej klasy znajdują się konkretne implementacje sił takich jak *Drag*, *Gravity* oraz *Thrust*, które są używane przez silnik do obliczeń. Więcej o implementacji tych sił opisano w podrozdziale 2.5.

2.4.4 Proces symulacji.

Proces symulacji fizycznej lotu rakiety jest wieloetapowy i składa się z dwóch głównych procesów: konfiguracji oraz przeprowadzenia symulacji. Poniżej opisane szczegółowo zostały oba te procesy.

2.4.5 Konfiguracja symulacji (Setup)

Zanim symulacja zostanie przeprowadzona, tworzony jest plik zawierający konfigurację symulacji. W tym pliku zapisywane są informacje o tym, jakie parametry powinna mieć symulacja.

Pierwszym krokiem jest stworzenie obiektów rakiety, środowiska oraz silnika fizycznego:

```

1 object = Rocket()
2 environment = Environment()
3 physics_engine = PhysicsEngine(object, environment, 0.1)

```

W tej części kodu tworzymy:

- `object` - instancję klasy `Rocket`, która reprezentuje raketę,
- `environment` - instancję klasy `Environment`, która definiuje środowisko symulacji,
- `physics_engine` - instancję klasy `PhysicsEngine`, która będzie zarządzać całą symulacją fizyczną.

Po utworzeniu silnika fizycznego, dodajemy do niego różne siły, które będą działać na raketę:

```

1 physics_engine.add_force(Forces.gravity)
2 physics_engine.add_force(Forces.thrust)
3 physics_engine.add_force(Forces.drag)

```

W tej części kodu dodajemy trzy siły:

- `Forces.gravity` - siła grawitacji,
- `Forces.thrust` - siła ciągu,
- `Forces.drag` - siła oporu powietrza.

Na końcu zapisujemy stan silnika fizycznego do pliku przy użyciu `pickle`:

```

1 config = Config()
2 path =
    os.path.join(config["SIMULATION"]["simulation_files_folder_path"],
        object.name)
3 save(physics_engine, path, "physics_engine.pkl")

```

W efekcie działania tego skryptu powstaje konfiguracja symulacji zawierająca informacje o rakiecie, środowisku i siłach, które powinny działać na raketę w trakcie trwania symulacji.

2.4.6 Symulacja

Po zapisaniu plików z konfiguracją, wczytujemy je do skryptu `simulation.py` i uruchamiamy symulację. W tym celu używamy poniższego kodu:

```

1 simulation_name = "name_of_simulation"
2
3 config = Config()

```

```

4 path =
    os.path.join(config["SIMULATION"]["simulation_files_folder_path"],
        simulation_name, "physics_engine.pkl")
5
6 with open(path, "rb") as file:
7     physics_engine = pickle.load(file)
8
9 physics_engine.start()

```

W pierwszej kolejności określamy nazwę symulacji, a następnie wczytujemy obiekt `physics_engine` z pliku. Wczytany silnik jest uruchamiany za pomocą metody `start()`, która inicjuje symulację. Funkcja ta:

- wczytuje wcześniej zapisany stan symulacji,
- rozpoczyna główną pętlę symulacyjną,
- zapisuje kolejne wyniki do pliku.

Działanie pętli symulacyjnej silnika fizycznego

Pętla symulacyjna silnika `PhysicsEngine` jest odpowiedzialna za zarządzanie całym przebiegiem symulacji. Działa w sposób iteracyjny, krok po kroku, aktualizując stan obiektów w symulacji w czasie rzeczywistym. Kluczowym elementem tego procesu jest kontrolowanie czasu symulacji oraz zarządzanie siłami działającymi na obiekty.

Główna pętla symulacyjna działa w następujący sposób:

1. **Aktualizacja czasu:** Po każdym kroku, zmieniane są dwie zmienne: `simulation_time` (całkowity czas symulacji) oraz `simulation_tick` (liczba iteracji).
2. **Obliczanie sił:** Silnik oblicza siły działające na obiekt poprzez wywołanie metod obliczających wynikową siłę i moment obrotowy. Zgodnie z wcześniej dodanymi siłami, dla każdego obiektu w symulacji obliczane są siły, które wpływają na jego ruch.
3. **Aktualizacja stanu obiektu:** Na podstawie obliczonych sił, aktualizowany jest stan obiektu: jego pozycja, prędkość, przyspieszenie oraz orientacja (rotacja). W tym kroku uwzględniane są zmiany zarówno w ruchu translacyjnym, jak i obrotowym.
4. **Sprawdzanie warunków zakończenia:** Pętla symulacyjna sprawdza, czy rakieta nie osiągnęła powierzchni ziemi (czyli gdy jej wysokość jest mniejsza od zera), lub czy nie osiągnięto maksymalnego czasu symulacji. Jeśli jeden z tych warunków jest spełniony, symulacja zostaje zakończona.

5. **Zapis danych:** Po każdej iteracji pętli, dane o stanie obiektu są zapisywane do pliku, aby umożliwić późniejszą analizę wyników symulacji.

Zakończenie symulacji

Symulacja kończy się, gdy rakieta dotknie ziemi lub gdy upłynie maksymalny czas symulacji. Wówczas silnik informuje użytkownika o zakończeniu symulacji. Wszystkie dane są zapisane, a wyniki są gotowe do analizy.

2.4.7 Uzasadnienie przyjętego rozwiązania

Implementacja wyżej przedstawionego rozwiązania pozwala na uzyskanie wielu benefitów, pomagających osiągnąć poczynione założenia. Są to między innymi:

Modułowość

Jednym z kluczowych założeń przy projektowaniu systemu była jego modułowość. Każda z głównych klas, takich jak `PhysicsEngine`, `Forces` czy `Rocket`, pełni dobrze zdefiniowaną rolę, co umożliwia łatwe modyfikowanie i rozwijanie projektu. Dzięki temu każdą część systemu można rozwijać niezależnie, bez wpływu na pozostałe komponenty. Pozwala to na współpracę między współautorami, jasność implementacji oraz łatwe testowanie i debugowanie.

Łatwość dodawania nowych sił

Projekt został zaprojektowany z myślą o prostocie rozszerzania o nowe siły. Obecność klasy `Forces` z różnymi zdefiniowanymi siłami, takimi jak `gravity`, `thrust` i `drag`, pozwala na łatwe dodanie nowych sił, np. oporu magnetycznego, sił aerodynamicznych specyficznych dla różnych kształtów rakiet, czy nawet sił wynikających z interakcji z innymi obiektami w symulacji.

Nowe siły mogą być łatwo zaimplementowane jako funkcje, które przyjmują obiekt rakiety i środowisko jako argumenty, co zapewnia spójność interfejsu. Ponadto mechanizm dodawania sił za pomocą metody `add_force` pozwala na ich dynamiczne przypisywanie i usuwanie w trakcie symulacji. Jeżeli nowe siły wymagają dodania do obiektu rakiety nowych parametrów, w łatwy sposób można rozwinąć jeden z istniejących interfejsów lub dodać zupełnie nowy interfejs umożliwiający tworzenie nowej klasy sił.

Łatwość zmiany parametrów

Kolejnym istotnym atutem tego rozwiązania jest możliwość łatwej zmiany parametrów fizycznych oraz konfiguracji symulacji. Wartości takie jak krok czasowy, maksymalny

czas symulacji, czy parametry sił (np. współczynniki oporu powietrza) mogą być modyfikowane w prosty sposób, bez konieczności ingerencji w kod samego silnika. Dzięki temu użytkownicy mogą łatwo dostosować symulację do swoich potrzeb, na przykład zmieniając warunki środowiskowe lub zachowanie rakiety. Przydatne jest to do wielokrotnego symulowania lotów rakiety w zmiennych warunkach. Sprzyja temu także możliwość tworzenia konfiguracji symulacyjnych.

Integracja z algorytmami sztucznej inteligencji

System jest przystosowany do wykorzystania w kontekście algorytmów sztucznej inteligencji. Dzięki temu, że cała symulacja jest oparta na obiektach i metodach z jasno zdefiniowanymi interfejsami, łatwo jest połączyć ją z systemami uczącymi się.

PhysicsEngine oraz **Rocket** dostarczają niezbędnych danych o ruchu obiektów (np. prędkości, przyspieszenia, pozycji), które mogą być wykorzystywane przez modele AI do dalszego uczenia się i optymalizacji działań.

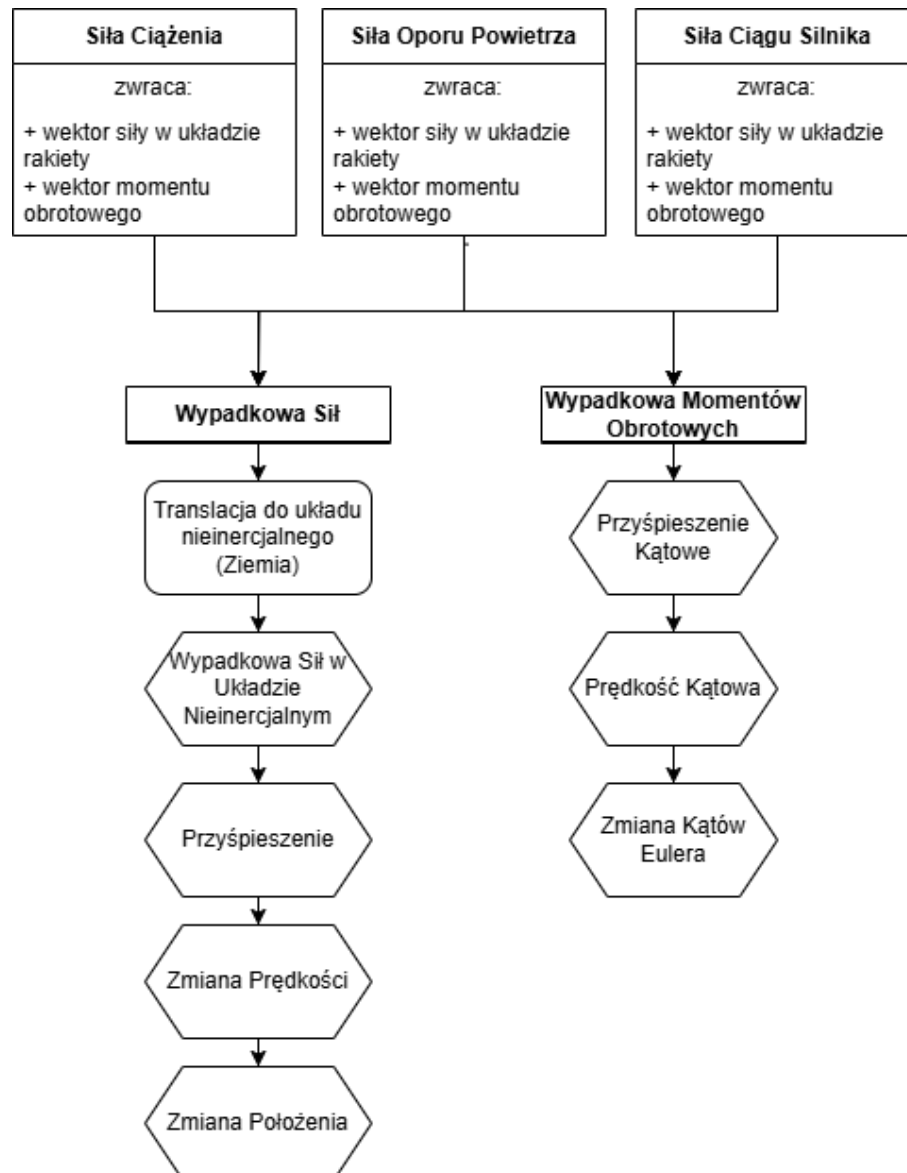
Ponadto, zapis i odczyt danych symulacji za pomocą plików **pickle** umożliwia łatwą wymianę danych między różnymi komponentami systemu, co jest istotne w kontekście przechowywania wyników symulacji i wykorzystywania ich w dalszych analizach.

Inną własnością ułatwiającą integrację z modelami ML jest krokowość silnika. Po każdym kroku (tick'u), model ma możliwość analizy stanu obiektów i zmiany dostępnych dla niego parametrów, co wpływa na stan obiektów w symulacji. W ten sposób silnik przystosowany jest do pracy w trybie uczenia nadzorowanego.

2.5 Model sił działających na raketę

Ważną częścią działanie silnika fizycznego jest implementacja sił fizycznych. W przypadku symulacji lotu rakiety, kluczowym elementem są siły, które oddziałują na raketę oraz sposób, w jaki owe siły są składane i jak wyliczany jest ich wpływ na ruch obiektu. W niniejszym podrozdziale opisane zostały te dwa kluczowe zagadnienia.

Na rysunku 2.2 przedstawiona została wysokopoziomowa architektura obliczeń sił w silniku fizycznym.



Rysunek 2.2. Schemat działania sił w symulacji

2.5.1 Funkcje pomocnicze

Funkcje pomocnicze stanowią podstawę obliczeń, umożliwiając między innymi transformację wektorów między lokalnym i globalnym układem współrzędnych. Transformacje te są kluczowe, ponieważ wiele sił, takich jak opór powietrza czy ciąg silnika, jest obliczanych w lokalnym układzie rakiety, a następnie przeliczanych do globalnego układu inercjalnego, w którym odbywają się końcowe obliczenia ruchu.

Funkcja tworząca macierz obrotu

Funkcja `euler_to_rotation_matrix` generuje macierz obrotu na podstawie kątów Eulera (ϕ_1 , ϕ_2 , ϕ_3), odpowiadających rotacjom wokół kolejno osi X, Y oraz Z.

```
1 def euler_to_rotation_matrix(angles):
2     fi1, fi2, fi3 = angles
3
4     Rx = np.array([
5         [1, 0, 0],
6         [0, np.cos(fi1), -np.sin(fi1)],
7         [0, np.sin(fi1), np.cos(fi1)]
8     ])
9
10    Ry = np.array([
11        [np.cos(fi2), 0, np.sin(fi2)],
12        [0, 1, 0],
13        [-np.sin(fi2), 0, np.cos(fi2)]
14    ])
15
16    Rz = np.array([
17        [np.cos(fi3), -np.sin(fi3), 0],
18        [np.sin(fi3), np.cos(fi3), 0],
19        [0, 0, 1]
20    ])
21
22    R = Rz @ Ry @ Rx
23    return R
```

Macierz obrotu jest wynikiem sekwencyjnego mnożenia macierzy rotacji R_x , R_y oraz R_z . Dzięki temu możliwe jest precyzyjne odwzorowanie orientacji rakiety w przestrzeni trójwymiarowej, co jest niezbędne do transformacji sił i prędkości.

Transformacja układów współrzędnych

Funkcje `transform_to_global` i `transform_to_local` służą do konwersji wektorów między lokalnym układem współrzędnych rakiety a globalnym układem inercyjnym.

```
1 def transform_to_global(local_vector, angles):
2     R = euler_to_rotation_matrix(angles)
3     return R @ local_vector
```

Funkcja `transform_to_global` używa macierzy rotacji `R`, aby przekształcić wektor z układu lokalnego do globalnego. Wektory lokalne, takie jak ciąg silnika, są orientowane zgodnie z orientacją rakiety w przestrzeni.

```
1 def transform_to_local(global_vector, angles):
2     R = euler_to_rotation_matrix(angles)
3     return R.T @ global_vector
```

Analogicznie, funkcja `transform_to_local` wykorzystuje transponowaną macierz rotacji, aby przekształcić wektor globalny na układ lokalny rakiety. Przykładem zastosowania jest obliczenie siły grawitacji w układzie lokalnym.

2.5.2 Siły działające na raketę

W symulacji uwzględniono pięć głównych sił: grawitację, opór powietrza, ciąg silnika, siłę nośną oraz siłę wiatru. Każda z tych sił jest obliczana na podstawie aktualnego stanu rakiety i środowiska, a ich wypadkowa wpływa na dynamikę ruchu rakiety.

Siła grawitacji

Siła grawitacji działa pionowo w dół, zgodnie z osią *Z* globalnego układu współrzędnych, i wynika z masy rakiety oraz przyspieszenia grawitacyjnego. Funkcja `gravity` oblicza wartość siły w układzie lokalnym rakiety, przekształcając ją z układu globalnego:

```
1 def gravity(object: IObject, environment: IEnvironment, time:
2     float):
3     global_gravity = np.array([0, 0, -object.mass *
4         Constants.GRAVITATIONAL_ACCELERATION.value], dtype=float)
5     local_gravity = transform_to_local(global_gravity,
6         object.rotation)
7     torque = np.array([0, 0, 0], dtype=float)
8     return local_gravity, torque
```

Opis działania funkcji:

- Obliczana jest wartość siły grawitacji w układzie globalnym jako iloczyn masy rakiety i przyspieszenia grawitacyjnego.
- Wektor siły grawitacji jest przekształcany na układ lokalny, aby uwzględnić aktualną orientację rakiety.
- Ponieważ siła grawitacji działa na środek masy, moment obrotowy wynosi zero.

Siła oporu powietrza

Siła oporu powietrza jest przeciwnie skierowana do prędkości rakiety względem otaczającego ją powietrza. Wartość siły jest proporcjonalna do kwadratu tej prędkości. Funkcja `drag` oblicza zarówno wartość siły, jak i moment obrotowy wynikający z jej działania:

```

1 def drag(object: IAerodependent, environment: IEnvironment, time:
    float):
2     if (object.velocity == np.array([0, 0, 0], dtype=float)).all():
3         return np.array([0, 0, 0], dtype=float), np.array([0, 0,
4             0], dtype=float)
5
6     Cd = object.drag_coefficient
7     relative_velocity = object.relative_velocity
8     reference_area = object.reference_area
9     drag_mag = 0.5 * environment.get_air_density() *
        np.linalg.norm(relative_velocity) ** 2 * Cd * reference_area
10    drag_v = -drag_mag * (relative_velocity /
        np.linalg.norm(relative_velocity))
11    drag_l = transform_to_local(drag_v, object.rotation)
12    torque = np.cross(drag_l, np.array([0, 0, -object.cop],
        dtype=float))
    return drag_v, torque

```

Opis działania funkcji:

- Wartość siły oporu jest obliczana na podstawie równania

$$F_d = \frac{1}{2} \rho v^2 C_d A,$$

gdzie ρ to gęstość powietrza, v to prędkość względna, C_d to współczynnik oporu, a A to powierzchnia przekroju poprzecznego.

- Kierunek siły oporu jest przeciwny do kierunku prędkości względnej rakiety względem powietrza.

- Siła i moment obrotowy są przeliczane na układ lokalny rakiety.

Siła ciągu silnika

Siła ciągu jest generowana przez spaliny wydostające się z silnika rakiety. Jej wartość zależy od masowego natężenia przepływu gazów oraz prędkości wylotowej. Funkcja `thrust` oblicza siłę ciągu i moment obrotowy:

```

1 def thrust(object: IThrust, environment: IEnvironment, time:
    float):
2     thrust_mag = object.mass_flow_rate(time) *
        object.exhaust_velocity(time)
3     thrust_v = np.array([0, 0, thrust_mag], dtype=float)
4     thrust_g = transform_to_global(thrust_v, object.engine_angle)
5     torque_l = np.cross(thrust_g, np.array([0, 0, -object.cot],
        dtype=float))
6     return thrust_g, torque_l

```

Opis działania funkcji:

- Wielkość siły ciągu jest obliczana jako iloczyn natężenia przepływu masy gazów i ich prędkości wylotowej.
- Wektor siły ciągu jest przekształcany z układu lokalnego (związanego z silnikiem) na układ globalny, uwzględniający aktualny kąt nachylenia silnika względem rakiety.
- Moment obrotowy wynika z przesunięcia środka ciągu (*Center of Thrust*, *CoT*) względem środka masy rakiety.

Siła nośna

Siła nośna powstaje w wyniku oddziaływania powietrza na powierzchnię rakiety, której kształt oraz orientacja wpływają na wartość tej siły. Jest obliczana na podstawie prędkości rakiety względem powietrza, współczynnika nośności oraz powierzchni, na którą działa ta siła. Funkcja `lift` oblicza siłę nośną oraz moment obrotowy:

```

1 def lift(object: IAerodependent, environment: IEnvironment, time:
    float) -> (
2     NDArray[np.float64], NDArray[np.float64]):
3     if (object.velocity == np.array([0, 0, 0], dtype=float)).all():
4         return np.array([0, 0, 0], dtype=float), np.array([0, 0,
        0], dtype=float)
5

```

```

6   Cl = object.lift_coefficient
7   relative_velocity = object.relative_velocity
8   reference_area = object.reference_area
9   air_density = environment.get_air_density()
10  lift_mag = 0.5 * air_density * np.linalg.norm(
11      relative_velocity) ** 2 * Cl * reference_area
12  velocity_unit = relative_velocity /
13      np.linalg.norm(relative_velocity)
14  lift_direction = np.cross(velocity_unit, np.array([0, 0, 1]))
15  if np.linalg.norm(lift_direction) == 0:
16      lift_direction = np.array([0, 1, 0])
17  lift_v = lift_mag * lift_direction /
18      np.linalg.norm(lift_direction)
19  lift_l = transform_to_local(lift_v, object.rotation)
20  torque = np.cross(lift_l, np.array([0, 0, -object.cop],
21      dtype=float))
22
23  return lift_l, torque

```

Opis działania funkcji:

- Siła nośna jest obliczana na podstawie wzoru:

$$L = 0.5 \cdot \rho \cdot v^2 \cdot A \cdot C_L,$$

gdzie ρ to gęstość powietrza, v to norma prędkości względnej rakiety, A to powierzchnia referencyjna rakiety, a C_L to współczynnik nośności.

- Wektor prędkości względnej rakiety jest normalizowany, aby uzyskać jednostkowy wektor prędkości (`velocity_unit`).
- Kierunek siły nośnej jest wyznaczany jako wektor prostopadły do prędkości rakiety, który jest obliczany za pomocą iloczynu wektorowego (`lift_direction`).
- Jeśli wektor siły nośnej ma zerową normę (co może się zdarzyć, gdy prędkość rakiety wynosi zero), przypisujemy domyślny kierunek wzdłuż osi Y (`[0, 1, 0]`).
- Siła nośna `lift_v` jest obliczana jako iloczyn wielkości siły nośnej (`lift_mag`) i znormalizowanego kierunku (`lift_direction`).
- Siła nośna i moment obrotowy są przekształcane na układ lokalny rakiety.

Siła wiatru

Siła wiatru działa na powierzchnię rakiety, zmieniając jej trajektorię oraz stabilność.

Funkcja `wind` oblicza wartość siły wiatru oraz moment obrotowy:

```
1 def wind(object: IAerodependent, environment: IEnvironment, time:
    float):
2     if (object.velocity == np.array([0, 0, 0], dtype=float)).all():
3         return np.array([0, 0, 0], dtype=float), np.array([0, 0,
4             0], dtype=float)
5
6     wind_v_global = environment.get_wind()
7     wind_v_local = transform_to_local(wind_v_global,
8         object.rotation)
9
10    torque = np.cross(wind_v_local, np.array([0, 0, -object.cop],
11        dtype=float))
12
13    return wind_v_local, torque
```

Opis działania funkcji:

- Siła wiatru w układzie globalnym jest pobierana z parametrów środowiska za pomocą metody `environment.get_wind()`.
- Siła ta jest następnie przekształcana na układ lokalny rakiety, uwzględniając jej aktualną orientację.
- Moment obrotowy jest obliczany na podstawie wektora siły wiatru oraz odległości od środka masy rakiety.

2.5.3 Obliczanie zmian w ruchu rakiety

Funkcja `compute_change` odpowiada za aktualizację parametrów dynamicznych rakiety, bazując na wypadkowych siłach i momentach pędu. Zawiera obliczenia zarówno dla ruchu postępowego, jak i obrotowego:

```
1 def compute_change(self, force: NDArray[np.float64], torque:
    NDArray[np.float64]):
2     global_force = transform_to_global(force, self.object.rotation)
3     self.object.acceleration = global_force / self.object.mass
4     velocity_change = self.object.acceleration * self.time_step
5     self.object.velocity += velocity_change
6
7     position_change = self.object.velocity * self.time_step
8     self.object.position += position_change
```

```

9
10 if isinstance(self.object, IAerodependent) and
    isinstance(self.object, IObject):
11     self.object.angle_of_attack =
        self.calculate_angle_of_attack()
12     self.object.relative_velocity = self.object.velocity -
        self.environment.get_wind()
13     self.object.drag_coefficient = (
14         C.DRAG_COEFFICIENT_PARALLEL.value *
            np.cos(self.object.angle_of_attack) ** 2
15         + C.DRAG_COEFFICIENT_PERPENDICULAR.value *
            np.sin(self.object.angle_of_attack) ** 2
16     )
17
18     self.object.angular_acceleration =
        np.linalg.inv(self.object.inertia_tensor) @ torque
19     self.object.angular_velocity +=
        self.object.angular_acceleration * self.time_step
20
21     wx, wy, wz = self.object.angular_velocity
22     phi, theta, psi = self.object.rotation
23
24     dot_phi = wx + (np.tan(theta) * np.sin(phi) * wy + np.cos(phi)
        * wz)
25     dot_theta = np.cos(phi) * wy - np.sin(phi) * wz
26     dot_psi = np.sin(phi) / np.cos(theta) * wy + np.cos(phi) * wz
27
28     self.object.rotation[0] += dot_phi * self.time_step
29     self.object.rotation[1] += dot_theta * self.time_step
30     self.object.rotation[2] += dot_psi * self.time_step

```

W ramach tej funkcji wykonywane są następujące operacje:

- **Obliczenia ruchu postępowego:** Na podstawie siły wypadkowej obliczane są przyspieszenie, zmiana prędkości oraz przesunięcie pozycji rakiety.
- **Obliczenia ruchu obrotowego:** Bazując na momencie pędu, wyznaczane są przyspieszenie kątowe, prędkość kątowna i zmiana orientacji rakiety w przestrzeni.
- **Korekcje aerodynamiczne:** Jeśli obiekt implementuje interfejs IAerodependent, wyliczane są dodatkowe parametry, takie jak kąt natarcia, względna prędkość wiatru czy współczynnik oporu aerodynamicznego.

Kąt natarcia rakiety

W trakcie aktualizacji stanu rakiety wywoływana jest funkcja `calculate_angle_of_attack`, która oblicza kąt natarcia, czyli kąt między wektorem prędkości rakiety a osią Z w układzie inercyjnym. Implementacja tej funkcji wygląda następująco:

```
1 def calculate_angle_of_attack(self):
2     object_velocity = self.object.velocity
3     wind_velocity = self.environment.get_wind()
4     relative_velocity = object_velocity - wind_velocity
5     relative_velocity_unit = relative_velocity /
6         np.linalg.norm(relative_velocity)
7
8     reference_axis_local = np.array([0, 0, 1], dtype=float)
9     reference_axis_global =
10         transform_to_global(reference_axis_local,
11                             self.object.rotation)
12
13     cos_theta = np.dot(relative_velocity_unit,
14                         reference_axis_global)
15     angle_of_attack = np.arccos(np.clip(cos_theta, -1.0, 1.0))
16
17     return angle_of_attack
```

Obliczanie kąta natarcia ma kluczowe znaczenie dla poprawnego modelowania sił aerodynamicznych, ponieważ wpływa bezpośrednio na wyznaczenie współczynnika oporu oraz siły nośnej działającej na rakietę.

Uwagi dotyczące układów współrzędnych

Warto zauważyć, że wszystkie obliczenia dotyczące ruchu postępowego są prowadzone w układzie nieinercyjnym związanym z Ziemią, podczas gdy obliczenia ruchu obrotowego wykonywane są w układzie współrzędnych związanym z rakietą. Dzięki temu symulacja uwzględnia złożoność dynamiki obiektów ruchomych w przestrzeni trójwymiarowej.

2.5.4 Podsumowanie

Przedstawiony powyżej model zaimplementowanej fizyki ruchu rakiety w symulacji lotu, jest modelem uproszczonym, nie biorącym pod uwagę wielu czynników. Spowodowane jest to niewystarczającymi zasobami (czasem, wiedzą, mocą obliczeniową). Model ten jednak dzięki modułowej architekturze silnika może być w łatwy sposób rozbudowany

i skomplikowany, tak aby bardziej realnie odzwierciedlał warunki panujące podczas rzeczywistych lotów rakietowych. Na potrzeby niniejszej pracy inżynierskiej zaimplementowana fizyka wydaje się być wystarczającą.

2.6 Moduł wizualizacyjny symulacji.

2.7 Dokładność symulacji.

2.8 Ewolucja projektu i wnioski.

3 Moduł II: Algorytm sterujący rakieta.

3.1 Teoretyczne podstawy.

3.2 Użyte technologie.

Do stworzenia algorytmu sterującego rakieta wykorzystane zostały następujące technologie:

- gówno
- dupa

3.3 Wybór odpowiedniego algorytmu uczenia maszynowego

3.4 Zbieranie danych treningowych.

3.5 Proces uczenia i optymalizacja.

3.6 Ewolucja projektu i wnioski.

4 Integracja komponentów: Silnik fizyczny i sztuczna inteligencja

4.1 Interakcja między silnikiem fizycznym a modelem AI

4.2 Synchronizacja działania obu modułów

4.3 Rozwój interfejsu użytkownika

5 Wyniki działania i wnioski

5.1 Ewaluacja działania silnika fizycznego

5.2 Ocena skuteczności modelu sztucznej inteligencji

5.3 Analiza osiągniętych rezultatów i ich implikacje

6 Podsumowanie

6.1 Podsumowanie osiągnięć

6.2 Wnioski końcowe

6.3 Perspektywy rozwoju i dalsze badania

Bibliografia

Wyrażam zgodę na udostępnienie mojej pracy w czytelniach Biblioteki SGGW
w tym w Archiwum Prac Dyplomowych SGGW.

.....
(czytelny podpis autora pracy)

